

Java Tutorial: Primitive Types, Wrapper Classes, and Pooling

1. Primitive Data Types in Java

Java has **8 primitive types** — simple, memory-efficient values.

Type	Size	Example	Description
byte	8-bit	byte b = 1;	Small whole numbers
short	16-bit	short s = 1000;	Larger than byte
int	32-bit	int i = 42;	Most common integer type
long	64-bit	long l = 123456L;	Very large numbers
float	32-bit	float f = 3.14f;	Single-precision decimal
double	64-bit	double d = 3.14;	Double-precision decimal
char	16-bit	char c = 'A';	Single character
boolean	1-bit	boolean b = true;	True/false

2. Wrapper Classes

Primitive types are **not objects**. Java provides **wrapper classes** to use primitives where objects are required (e.g., in collections).

Primitive	Wrapper
int	Integer
double	Double
char	Character
boolean	Boolean
...	...

Example:

```
int x = 10;
Integer y = Integer.valueOf(x); // Explicit boxing
List<Integer> list = new ArrayList<>();
list.add(x); // Autoboxing
```

3. Autoboxing and Unboxing

Java can automatically convert between primitive types and their wrapper classes:

```
Integer obj = 42;      // autoboxing (int → Integer)
int num = obj;         // unboxing (Integer → int)
```

◆ 4. Integer Pooling

🔄 What is Integer Pooling?

Java **caches** **Integer** values from **-128 to 127** to improve performance.

```
Integer a = 100;
Integer b = 100;
System.out.println(a == b); // true – both point to same pooled object

Integer x = 200;
Integer y = 200;
System.out.println(x == y); // false – different objects
```

! Use `.equals()` to compare values of wrapper objects:

```
System.out.println(x.equals(y)); // true
```

◆ 5. String Pooling

Strings behave **differently** because they are **immutable** and pooled **by default** when created using literals.

```
String s1 = "hello";
String s2 = "hello";
System.out.println(s1 == s2); // true – both point to pooled string
```

But if you create a string using `new String()`, it creates a **new object**:

```
String s3 = new String("hello");
System.out.println(s1 == s3); // false – different objects
```

Use `.equals()` for value comparison:

```
System.out.println(s1.equals(s3)); // true
```

◆ Key Differences Between Integer & String Pooling

Aspect	Integer Pooling	String Pooling
Pooling Range	Only <code>-128</code> to <code>127</code>	All string literals
Pooling Method	<code>Integer.valueOf()</code>	Implicit via literals
Affected Type	Only <code>Integer</code>	All string literals
Custom Pooling	Not supported	Use <code>intern()</code> manually

📌 Summary

- Use **primitive types** for performance.
- Use **wrapper classes** with collections or generics.
- Java pools **small Integers** (`-128` to `127`) and **string literals**.
- Always use `.equals()` for comparing wrapper or string values.
- Be cautious with object identity (`==`) vs. value equality.